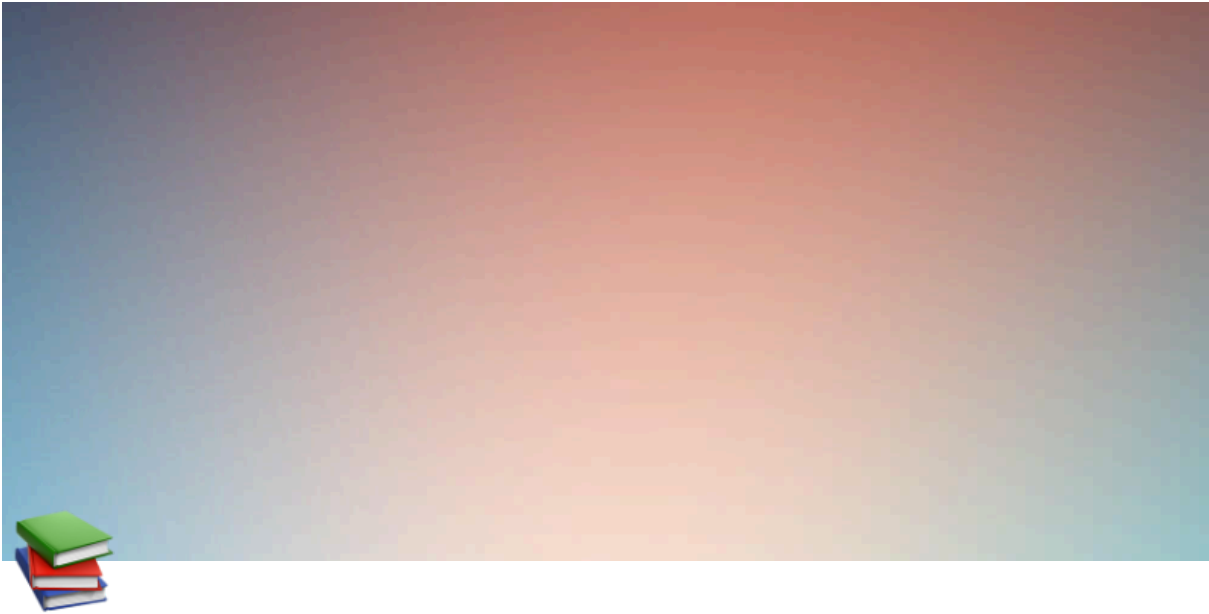




Ch.8 군집화

- NLP이나 텍스트 분석이나?

NLP : 머신이 인간의 언어를 이해하고 해석하는 데 중점 두고 기술 발전

텍스트 분석 향상 기반 기술

Text Mining : 비정형 텍스트에서 의미 있는 정보 추출

비즈니스 인텔리전스나 예측 분석 등의 분석 작업 수행

텍스트 분류(Text Classification): Text Categorization이라고도 합니다. 문서가 특정 분류 또는 카테고리에 속하는 것을 예측하는 기법을 통칭합니다. 예를 들어 특정 신문 기사 내용이 연예/정치/사회/문화 중 어떤 카테고리에 속하는지 자동으로 분류하거나 스팸 메일 검출 같은 프로그램이 이에 속합니다. 지도학습을 적용합니다.

감성 분석(Sentiment Analysis): 텍스트에서 나타나는 감정/판단/믿음/의견/기분 등의 주관적인 요소를 분석하는 기법을 총칭합니다. 소셜 미디어 감정 분석, 영화나 제품에 대한 긍정 또는 리뷰, 여론조사 의견 분석 등의 다양한 영역에서 활용됩니다. Text Analytics에서 가장 활발하게 사용되고 있는 분야입니다. 지도학습 방법뿐만 아니라 비지도학습을 이용해 적용할 수 있습니다.

텍스트 요약(Summarization): 텍스트 내에서 중요한 주제나 중심 사상을 추출하는 기법을 말합니다. 대표적으로 토픽 모델링(Topic Modeling)이 있습니다.

텍스트 군집화(Clustering)와 유사도 측정: 비슷한 유형의 문서에 대해 군집화를 수행하는 기법을 말합니다. 텍스트 분류를 비지도학습으로 수행하는 방법의 일환으로 사용될 수 있습니다. 유사도 측정 역시 문서들간의 유사도를 측정해 비슷한 문서끼리 모을 수 있는 방법입니다.

01 텍스트 분석 이해

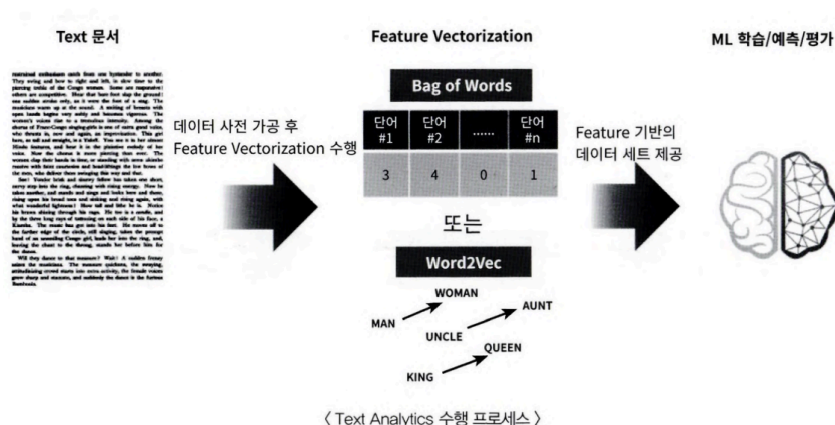
텍스트 분석: 비정형 데이터인 텍스트 분석

피처 벡터화 : 텍스트를 word 기반의 다수의 피처로 추출 → 피처에 단어 빈도수와 같은 숫자 값 부여 → 텍스트를 단어의 조합인 벡터값으로 표현

⇒ BOW(Bag of Words) & Word2Vec

▼ 텍스트 분석 수행 프로세스

1. 텍스트 사전 준비작업(텍스트 전처리): 텍스트를 피처로 만들기 전에 미리 클렌징, 대/소문자 변경, 특수문자 삭제 등의 클렌징 작업, 단어(Word) 등의 토큰화 작업, 의미 없는 단어(Stop word) 제거 작업, 어근 추출(Stemming/Lemmatization) 등의 텍스트 정규화 작업을 수행하는 것을 통칭합니다.
2. 피처 벡터화/추출: 사전 준비 작업으로 가공된 텍스트에서 피처를 추출하고 여기에 벡터 값을 할당합니다. 대표적인 방법은 BOW와 Word2Vec이 있으며, BOW는 대표적으로 Count 기반과 TF-IDF 기반 벡터화가 있습니다.
3. ML 모델 수립 및 학습/예측/평가: 피처 벡터화된 데이터 세트에 ML 모델을 적용해 학습/예측 및 평가를 수행합니다.



▼ 파이썬 기반의 NLP, 텍스트 분석 패키지

NLTK(Natural Language Toolkit for Python): 파이썬의 가장 대표적인 NLP 패키지입니다. 방대한 데이터 세트와 서브 모듈을 가지고 있으며 NLP의 거의 모든 영역을 커버하고 있습니다. 많은 NLP 패키지가 NLTK의 영향을 받아 작성되고 있습니다. 수행 속도 측면에서 아쉬운 부분이 있어서 실제 대량의 데이터 기반에서는 제대로 활용되지 못하고 있습니다.

Gensim: 토픽 모델링 분야에서 가장 두각을 나타내는 패키지입니다. 오래전부터 토픽 모델링을 쉽게 구현할 수 있는 기능을 제공해 왔으며, Word2Vec 구현 등의 다양한 신기능도 제공합니다. SpaCy와 함께 가장 많이 사용되는 NLP 패키지입니다.

SpaCy: 뛰어난 수행 성능으로 최근 가장 주목을 받는 NLP 패키지입니다. 많은 NLP 애플리케이션에서 SpaCy를 사용하는 사례가 늘고 있습니다.

- 파이썬에 '어근 처리' 라이브러리는 X
- 보통 NLTK/Gensim/SpaCy 와 결합해 애플리케이션 작성

02 텍스트 사전 준비 작업(텍스트 전처리) - 텍스트 정규화

- 텍스트 자체를 바로 피쳐 X
- **텍스트 정규화** : 텍스트를 머신러닝 알고리즘이나 NLP 애플리케이션에 입력 데이터로 사용하기 위해 클렌징, 정제, 토큰화, 어근화 등 다양한 텍스트 데이터 사전 작업 수행하는 것

▼ 클렌징

: 텍스트에서 분석에 방해 되는 불필요한 문자, 기호 등을 사전에 제거하는 작업

Ex) HTML, XML 태그나 특정 기호를 사전에 제거

▼ 텍스트 토큰화

- 토큰화의 유형 : 문장 토큰화 / 단어 토큰화
- NLTK가 다양한 API 제공

1. 문장 토큰화

- 문장의 마침표(.), 개행문자(\n) 등 문장의 마지막을 뜻하는 기호에 따라 분리
- 정규 표현식에 따른 문장 토큰화 가능

2. 단어 토큰화

- 문장을 단어로 토큰화
- 공백, 콤마(,), 마침표(.), 개행문자 등으로 단어 분리
- 정규 표현식에 따른 토큰화 가능

▼ 스톱 워드 제거

- 스톱 워드(Stop word) : 분석에 큰 의미 없는 단어 지칭
- 문법적인 특성으로 인해 빈번한 등장 → 중요한 단어로 인지 가능! → 의미 X 단어 제거 중요

▼ Stemming과 Lemmatization

- 문법적 또는 의미적으로 변화하는 단어의 원형 찾기

1. Stemming

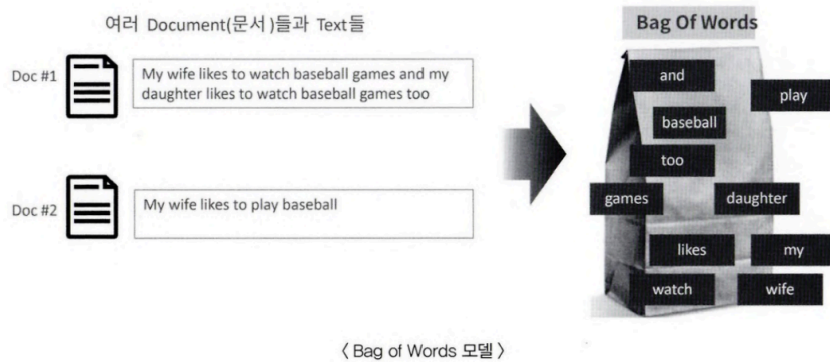
- 원형 단어로 변환 시 일반적인 / 단순화된 방법 적용 → 원래 단어에서 일부 철자 훼손된 어근 단어 추출
- NLTK - Porter, Lancaster, Snowball Stemmer

2. Lemmatization

- 품사와 같은 문법적인 요소와 의미적인 부분 감안 → 정확한 철자로 된 어근 단어 찾아줌
- Stemming보다 변환에 더 오랜 시간 / 정교
- NLTK - WordNetLemmatizer

03 Bag of Words - BOW

- 문서가 가지는 모든 단어를 문맥이나 순서 무시라고 일괄적으로 단어에 대해 빈도 값 부여해 피쳐 값 추출하는 모델
- 한꺼번에 봉투(Bag) 안에 넣은 뒤 흔들어서 섞는다



문장 1:

‘My wife likes to watch baseball games and my daughter likes to watch baseball games too’

문장 2:

‘My wife likes to play baseball’

1. 문장 1과 문장 2에 있는 모든 단어에서 중복을 제거하고 각 단어(feature 또는 term)를 칼럼 형태로 나열합니다. 그리고 나서 각 단어에 고유의 인덱스를 다음과 같이 부여합니다.
 ‘and’:0, ‘baseball’:1, ‘daughter’:2, ‘games’:3, ‘likes’:4, ‘my’:5, ‘play’:6, ‘to’:7, ‘too’:8, ‘watch’:9, ‘wife’:10
2. 개별 문장에서 해당 단어가 나타나는 횟수(Occurrence)를 각 단어(단어 인덱스)에 기재합니다. 예를 들어 baseball은 문장 1, 2에서 총 2번 나타나며, daughter는 문장 1에서만 1번 나타납니다.

	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Index 6	Index 7	Index 8	Index 9	Index 10
	and	baseball	daughter	games	likes	my	play	to	too	watch	wife
문장 1	1	2	1	2	2	2		2	1	2	1
문장 2		1			1	1	1	1			1

→ 문장 1에서 baseball은 2회 나타남

[BOW 모델 장점]

- 쉽고 빠른 구축
- 문서의 특징 잘 나타냄 - 여러 분야에 활용

[BOW 모델 단점]

문맥 의미(Semantic Context) 반영 부족: BOW는 단어의 순서를 고려하지 않기 때문에 문장 내에서 단어의 문맥적인 의미가 무시됩니다. 물론 이를 보완하기 위해 n -gram 기법을 활용할 수 있지만, 제한적인 부분에 그치므로 언어의 많은 부분을 차지하는 문맥적인 해석을 처리하지 못하는 단점이 있습니다.

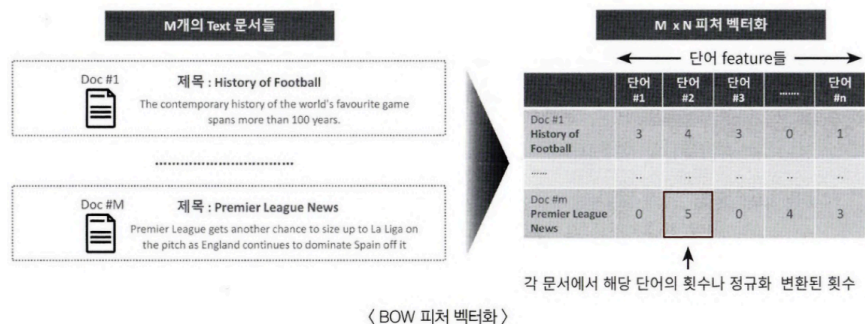
희소 행렬 문제(희소성, 희소 행렬): BOW로 피쳐 벡터화를 수행하면 희소 행렬 형태의 데이터 세트가 만들어지기 쉽습니다. 많은 문서에서 단어를 추출하면 매우 많은 단어가 칼럼으로 만들어집니다. 문서마다 서로 다른 단어로 구성되기에 단어가 문서마다 나타나지 않는 경우가 훨씬 더 많습니다. 즉, 매우 많은 문서에서 단어의 총 개수는 수만 ~ 수십만 개가 될 수 있는데, 하나의 문서에 있는 단어는 이 중 극히 일부뿐이므로 대부분의 데이터는 0 값으로 채워지게 됩니다. 이처럼 대규모의 칼럼으로 구성된 행렬에서 대부분의 값이 0으로 채워지는 행렬을 희소 행렬(Sparse Matrix)이라고 합니다. 이와는 반대로 대부분의 값이 0이 아닌 의미 있는 값으로 채워져 있는 행렬을 밀집 행렬(Dense Matrix)이라고 합니다. 희소 행렬은 일반적으로 ML 알고리즘의 수행 시간과 예측 성능을 떨어뜨리기 때문에 희소 행렬을 위한 특별한 기법이 마련되었습니다.

▼ BOW 피쳐 벡터화

- 텍스트를 특정 의미를 가지는 숫자형 값인 벡터 값으로 변환
- 각 문서의 텍스트를 단어로 추출해 피쳐로 할당 → 각 단어의 발생 빈도와 같은 값을 피쳐에 값으로 부여 → 각 문서를 단어 피쳐의 발생 빈도 값으로 구성된 벡터로 만드는 기법
- 기존 텍스트 데이터를 또 다른 형태의 피쳐 조합으로 변경 → 넓은 피쳐 추출에 포함

[BOW 모델에서의 피쳐 벡터화 수행]

- 모든 문서에서 모든 단어를 칼럼 형태로 나열 → 각 문서에서 해당 단어의 횟수나 정규화된 빈도를 값으로 부여하는 데이터 세트 모델로 변경



[BOW의 피쳐 벡터화 방식]

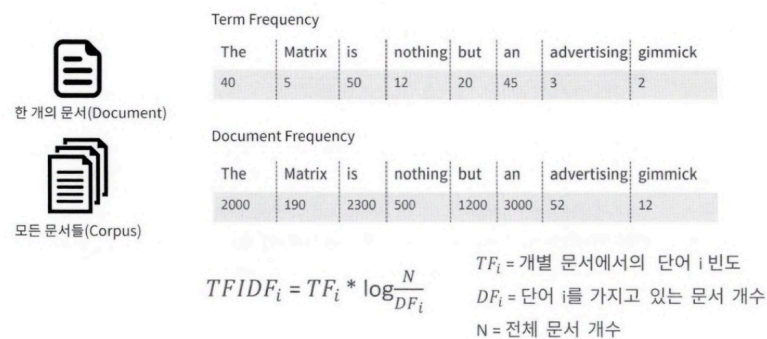
1. 카운트 기반의 벡터화

- 카운트 벡터화 : 단어 피처에 값 부여할 때 Count 부여하는 경우
- 카운트 값이 높을수록 → 중요한 단어!
- But, 카운트만 부여할 경우, 자주 사용되는 단어에도 높은 값 부여 가능

⇒ TF-IDF(Term Frequency Inverse Document Frequency)

2. TF-IDF(Term Frequency Inverse Document Frequency)

- 개별 문서에서 자주 나타나는 단어에 가중치
- 모든 문서에서 전반적으로 자주 사용되는 단어에는 패널티



▼ 사이킷런의 Count 및 TF-IDF 벡터화 구현: CountVectorizer, TfidfVectorizer

- 클래스 카운트 기반의 벡터화 구현한 클래스
- CountVectorizer 클래스 - 피처 벡터화 + 소문자 일괄 변환, 토큰화, 스톱 워드 필터링 등의 텍스트 전처리

[CountVectorizer 입력 파라미터]

파라미터 명	파라미터 설명
max_df	전체 문서에 걸쳐서 너무 높은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터입니다. 너무 높은 빈도수를 가지는 단어는 스톱 워드와 비슷한 문법적인 특성으로 반복적인 단어일 가능성이 높기에 이를 제거하기 위해 사용됩니다. max_df = 100과 같이 정수 값을 가지면 전체 문서에 걸쳐 100개 이하로 나타나는 단어만 피처로 추출합니다. Max_df = 0.95와 같이 부동소수점 값(0.0 ~ 1.0)을 가지면 전체 문서에 걸쳐 빈도수 0~95%까지의 단어만 피처로 추출하고 나머지 상위 5%는 피처로 추출하지 않습니다.
min_df	전체 문서에 걸쳐서 너무 낮은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터입니다. 수백~수천 개의 전체 문서에서 특정 단어가 min_df에 설정된 값보다 적은 빈도수를 가진다면 이 단어는 크게 중요하지 않거나 가비지(garbage)성 단어일 확률이 높습니다. min_df = 2와 같이 정수 값을 가지면 전체 문서에 걸쳐서 2번 이하로 나타나는 단어는 피처로 추출하지 않습니다. min_df = 0.02와 같이 부동소수점 값(0.0 ~ 1.0)을 가지면 전체 문서에 걸쳐서 하위 2% 이하의 빈도수를 가지는 단어는 피처로 추출하지 않습니다.
max_features	추출하는 피처의 개수를 제한하며 정수로 값을 지정합니다. 가령 max_features = 2000으로 지정할 경우 가장 높은 빈도를 가지는 단어 순으로 정렬해 2000개까지만 피처로 추출합니다.
stop_words	'english'로 지정하면 영어의 스톱 워드로 지정된 단어는 추출에서 제외합니다.
n_gram_range	Bag of Words 모델의 단어 순서를 어느 정도 보장하기 위한 n_gram 범위를 설정합니다. 튜플 형태로 (범위 최솟값, 범위 최댓값)을 지정합니다. 예를 들어 (1, 1)로 지정하면 토큰화된 단어를 1개씩 피처로 추출합니다. (1, 2)로 지정하면 토큰화된 단어를 1개씩(minimum 1), 그리고 순서대로 2개씩(maximum 2) 묶어서 피처로 추출합니다.
analyzer	피처 추출을 수행한 단위를 지정합니다. 당연히 디폴트는 'word'입니다. Word가 아니라 character의 특정 범위를 피처로 만드는 특정한 경우 등을 적용할 때 사용됩니다.
token_pattern	토큰화를 수행하는 정규 표현식 패턴을 지정합니다. 디폴트 값은 '\b\w+\b'로, 공백 또는 개행 문자 등으로 구분된 단어 분리자(\b) 사이의 2문자문자 또는 숫자, 즉 영숫자) 이상의 단어(word)를 토큰으로 분리합니다. analyzer= 'word'로 설정했을 때만 변경 가능하나 디폴트 값을 변경할 경우는 거의 발생하지 않습니다.
tokenizer	토큰화를 별도의 커스텀 함수로 이용시 적용합니다. 일반적으로 CountTokenizer 클래스에서 어근 변환 시 이를 수행하는 별도의 함수를 tokenizer 파라미터에 적용하면 됩니다.



▼ BOW 벡터화를 위한 희소 행렬

- 사이킷런 CountVectorizer/TfidfVectorizer 이용 → 텍스트를 피처 단위로 벡터화해 전환하고 CSR 형태의 희소 행렬 반환

← 수십만 개의 칼럼 →

	단어 1	단어 2	단어 3		단어 1000		단어 2000		단어 10000		단어 20000		단어 100000
문서1	1	2	2	0	0	0	0	0	0	0	0	0	0
문서2	0	0	1	0	0	1	0	0	1	0	0	0	1
문서 ...													
문서 10000	0	1	3	0	0	0	0	0	0	0	0	0	0

수천~수만 개 레코드

BOW의 Vectorization 모델은 너무 많은 0값이 메모리 공간에 할당되어 많은 메모리 공간이 필요하며 연산 시에도 데이터 액세스를 위한 많은 시간이 소모됩니다.

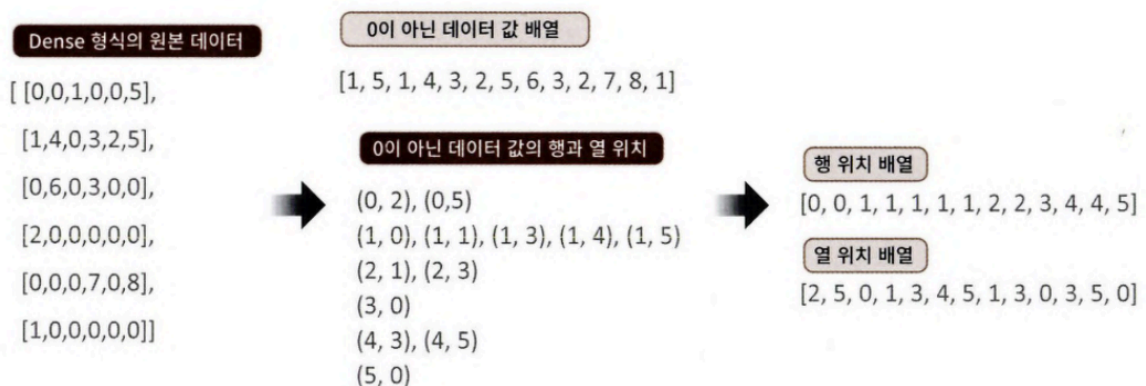
- 희소 행렬 - 너무 많은 불필요한 0 값이 메모리 공간에 할당 → 메모리 공간 많이 필요 → 행렬의 크기가 커서 연산 시에 데이터 액세스 위한 시간 많이 소모
- 물리적으로 적은 메모리 공간 차지할 수 있도록 변환
- COO & CSR

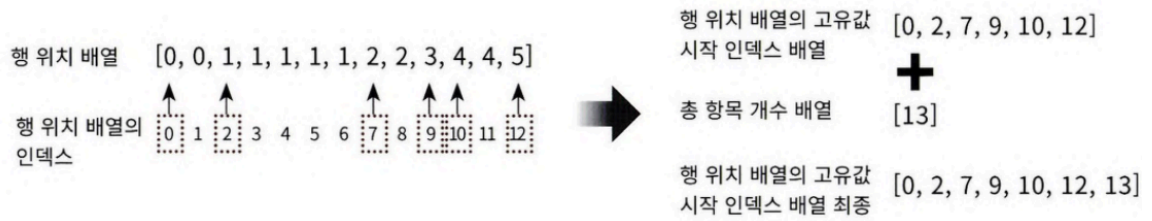
▼ 희소 행렬 - COO 형식

- COO(Coordinate : 좌표) 형식 : 0이 아닌 데이터만 별도의 데이터 배열(Array)에 저장, 그 데이터가 가리키는 행과 열의 위치를 별도의 배열로 저장하는 방식

▼ 희소 행렬 - CSR 형식

- CSR(Compressed Sparse Row) 형식 : COO 형식이 행과 열의 위치를 나타내기 위해 반복적인 위치 데이터를 사용해야 하는 문제점 해결한 방식





05 감성 분석

▼ 감성 분석 소개

- 문서의 주관적인 감성/의견/감정/기분 등을 파악하기 위한 방법
- 문서 내 텍스트가 나타내는 여러 가지 주관적인 단어와 문맥을 기반으로 감성 수치를 계산하는 방법 이용
- 긍정 감성 지수 / 부정 감성 지수 → 지수를 합산해 긍정 감성 / 부정 감성 결정

지도학습은 학습 데이터와 타깃 레이블 값을 기반으로 감성 분석 학습을 수행한 뒤 이를 기반으로 다른 데이터의 감성 분석을 예측하는 방법으로 일반적인 텍스트 기반의 분류와 거의 동일합니다.

비지도학습은 'Lexicon'이라는 일종의 감성 어휘 사전을 이용합니다. Lexicon은 감성 분석을 위한 용어와 문맥에 대한 다양한 정보를 가지고 있으며, 이를 이용해 문서의 긍정적, 부정적 감성 여부를 판단합니다.

▼ 지도학습 기반 감성 분석 실습 - IMDB 영화평

- IMDB 영화 사이트의 영화평 이용
- 영화평의 텍스트 분석 - 감정 분석 결과가 긍정 또는 부정인지 예측하는 모델 생성

▼ 비지도학습 기반 감성 분석 소개

- 비지도 감성 분석 : Lexion 기반
- 결정된 레이블 값 X 경우 유용

- 주로 감성만을 분석하기 위해 지원하는 감성 어휘 사전
- 감성 사전 : 긍정 감성 or 부정 감성 정도를 의미하는 수치
 - ↳ 감성 지수(Polarity score)
- 단어의 위치나 주변 단어, 문맥, POS(Part of Speech) 참고해 결정

[NLTK 포함한 대표적인 감성 사전]

SentiWordNet: NLTK 패키지의 WordNet과 유사하게 감성 단어 전용의 WordNet을 구현한 것입니다. WordNet의 Synset 개념을 감성 분석에 적용한 것입니다. WordNet의 Synset별로 3가지 감성 점수(sentiment score)를 할당합니다. 긍정 감성 지수, 부정 감성 지수, 객관성 지수가 그것입니다. 긍정 감성 지수는 해당 단어가 감성적으로 얼마나 긍정적인가를, 부정 지수는 얼마나 감성적으로 부정적인가를 수치로 나타낸 것입니다. 객관성 지수는 긍정/부정 감성 지수와 완전히 반대되는 개념으로 단어가 감성과 관계없이 얼마나 객관적인지를 수치로 나타낸 것입니다. 문장별로 단어들의 긍정 감성 지수와 부정 감성 지수를 합산하여 최종 감성 지수를 계산하고 이에 기반해 감성이 긍정인지 부정인지를 결정합니다.

VADER: 주로 소셜 미디어의 텍스트에 대한 감성 분석을 제공하기 위한 패키지입니다. 뛰어난 감성 분석 결과를 제공하며, 비교적 빠른 수행 시간을 보장해 대용량 텍스트 데이터에 잘 사용되는 패키지입니다.

Pattern: 예측 성능 측면에서 가장 주목받는 패키지입니다. 아쉽게도 현재 기준으로 파이썬 3.X 버전에서 호환이 되지 않고, 파이썬 2.X 버전에서만 동작합니다. 이 책에서는 사용 예제를 소개하지는 않습지만, 감성 분석에 관심이 많은 사람이라면 적용해 보는 것도 좋습니다.

▼ SentiWordNet을 이용한 감성 분석

1. WordNet Synset과 SentiWordNet SentiSynset

- POS(Part of Speech), 정의(Definition), 부명제(Lemma) 등으로 시맨틱적인 요소를 표현

2. SentiWordNet을 이용한 영화 감상평 감성 분석

1. 문서(Document)를 문장(Sentence) 단위로 분해
2. 다시 문장을 단어(Word) 단위로 토큰화하고 품사 태깅
3. 품사 태깅된 단어 기반으로 synset 객체와 senti_synset 객체를 생성
4. senti_synset 객체에서 긍정 감성/부정 감성 지수를 구하고 이를 모두 합산해 특정 임계치 값 이상일 때 긍정 감성으로, 그렇지 않을 때는 부정 감성으로 결정

▼ VADER를 이용한 감성 분석

- **VADER** : 소셜 미디어의 감성 분석 용도로 만들어진 룰 기반의 Lexicon
- SentimentIntensityAnalyzer 클래스 이용 → 감성 분석 제공